

00 教程内容介绍

一、教程内容概述

①第一部分

- 介绍 Godot 引擎基本开发思想
- 教授基本的代码入门和编程思想
- 制作一个简单的游戏

②第二部分

- 讲解 Godot 游戏的启动与运行机制。
- 第二部分强推有一定基础的 Godot 初学者进行观看。

③第三部分

- 讲解 2D 开发相关的 Node2D 类节点
- 讲解游戏界面开发相关的 Control 节点
- 补充介绍一些常用的节点

④第四部分

- 开发一个小型的 2D 游戏
- 归纳总结前几部分知识点

二、学习建议

- 多动手，要跟随课程中的案例敲击代码或操作引擎。
- 不必过多纠结于教程中提到的名词名字。而是要理解这些名词的作用。

三、入门检验标准

- 自己计划复刻一款简单的 2D 游戏，可以在程序层面自主完成一次从设计到制作的游戏开发过程。

01 游戏引擎介绍

一、游戏引擎的出现

在最早期的游戏开发中，并不存在游戏引擎这种软件，开发者们往往每开始一次项目就要重头开始编写一次代码。而随着后世游戏的发展，设备性能的不不断提升，游戏的内容也逐渐向现实世界靠拢，某些不同的游戏往往会出现类似的游戏元素。比如 FC 中的马里奥、魂斗罗、洛克人、恶魔城等都拥有 2D 平台跳跃的元素。



在这种情况下，开发者意识到他们不必再从零开始为不同的游戏编写代码，而是可以将描述游戏共性的代码多次利用，形成某类游戏的开发工具，这也就是我们常说的游戏引擎。而游戏引擎的出现，则大大简化了游戏程序开发的流程，也大幅度降低了游戏开发的难度。

二、现代游戏引擎的特征

- ①游戏引擎内置了描述游戏共性的代码，使得开发者不必纠结于如何从底层实现游戏内容。
- ②游戏引擎具有游戏内容预览功能，可以可视化的修改一些游戏数据。加快了工作效率。

三、学习游戏引擎要掌握的内容

- ①大致了解游戏引擎的开发思想。
- ②了解如何使用引擎内置编码来编写自己期望的游戏内容。
- ③了解熟悉引擎内部操作。（多操作多练习观摩他人视频）

01 游戏引擎介绍

一、游戏引擎的出现

在最早期的游戏开发中，并不存在游戏引擎这种软件，开发者们往往每开始一次项目就要重头开始编写一次代码。而随着后世游戏的发展，设备性能的不不断提升，游戏的内容也逐渐向现实世界靠拢，某些不同的游戏往往会出现类似的游戏元素。比如 FC 中的马里奥、魂斗罗、洛克人、恶魔城等都拥有 2D 平台跳跃的元素。



在这种情况下，开发者意识到他们不必再从零开始为不同的游戏编写代码，而是可以将描述游戏共性的代码多次利用，形成某类游戏的开发工具，这也就是我们常说的游戏引擎。而游戏引擎的出现，则大大简化了游戏程序开发的流程，也大幅度降低了游戏开发的难度。

二、现代游戏引擎的特征

- ①游戏引擎内置了描述游戏共性的代码，使得开发者不必纠结于如何从底层实现游戏内容。
- ②游戏引擎具有游戏内容预览功能，可以可视化的修改一些游戏数据。加快了工作效率。

三、学习游戏引擎要掌握的内容

- ①大致了解游戏引擎的开发思想。
- ②了解如何使用引擎内置编码来编写自己期望的游戏内容。
- ③了解熟悉引擎内部操作。（多操作多练习观摩他人视频）

02 Godot 引擎学习介绍

一、学习游戏引擎要掌握的内容

- ①大致了解游戏引擎的**开发思想**与工作流程。
- #②了解如何使用引擎内置编码来编写自己期望的游戏内容。
- #③了解熟悉引擎内部操作。（多操作多练习观摩他人视频）

二 Godot 的开发思想——围绕节点

①节点的特征与优势

- 节点是 Godot 内最常用最基本的开发组件
- 大部分节点都具有非常具体的功能,它们往往直接涉及人类的视听感官,如显示图片、播放音乐、显示模型、模拟物理体等。
- 节点之间可以用上下级的方式组合,一个节点可以有多个子节点。
- 节点的开发思想与人类相接近

②节点是在场景树与服务器的控制下运行的

- 场景树是游戏的主循环对象,只有在“树”下的节点才可以正常行使节点的主要功能。（节点初始化时并不在“树”下）
- “服务器”指的是 Godot 内置的各类 Server,节点的主要功能都要使用 server 实现,节点本身不具备实际功能。二者通过场景树进行协调沟通。
- “服务器”的编码比较接近底层,保障了游戏运行的效率。节点的设计思路比较接近人类,加速了游戏开发者开发游戏的效率。场景树起到了协调二者的作用。

③节点的组织方式——场景

- 场景是若干节点的集合
- 场景文件是记录若干节点集合的文件
- 游戏实际运行时不存在“场景”,场景是节点在文件系统中储存和加载的单位。
- 游戏从主场景开始运行。

④Godot 软件的界面介绍——围绕节点

- 1 场景栏:显示场景内节点,可以在这里增删一个场景内的节点。（还可以在这里为节点附加脚本。）
- 2 文件系统:显示场景文件与游戏素材文件。
- 3 视图区:预览场景内的可视节点。（编写代码与查看素材库）

- 4 检查器：显示节点的属性。（显示节点的信号连接、节点组情况与游戏素材文件属性）

三、GDScript——Godot 的原生语言

①GDScript 入门介绍

- GDScript 是 Godot 的原生语言，是最适合 Godot 开发的语言。
- 在入门阶段，我们只需要知道 GDScript 可以为节点拓展功能即可。

②GDScript 与节点的关系

- 节点对电脑而言具有非常具体的功能，可以令游戏程序执行显示图片、播放音频、模拟物理体等功能，但也仅限于此。
- 游戏除了上述的具体功能外，往往还需要具有一套内在的逻辑。就像我们现实中下的象棋，棋子本身只是普通的木头和石头，只是能让我们看得见“象棋”，而要让这些木头和石头能勾起人的兴趣，我们还要制定一套象棋的规则，规定这些棋子在何种情况下如何行走。
- GDS 负责实现内在逻辑，同时控制节点展示具体的功能。比如一个角色被一个怪物攻击掉血死亡的功能，除了要在 GDS 中进行角色生命值的削减和角色存活的判断，还要在判断角色死亡后控制节点播放死亡动画和音效。

③学习 GDScript 的步骤

- 了解掌握简单变量的定义与简单使用
- 了解掌握函数的定义、参数、返回值概念，认识常见函数
- 了解掌握 if 以及 while 的使用
- 了解掌握数组与 for 的简单使用

03 GDScript 简单入门——变量

一、变量

①变量的概念与简单操作

- 变量是可以代表数据的一种抽象概念。可以通过对变量名的操作来对变量代表的数据进行操作。
- 在 GDS 中，你可以创建自定义的变量，也可以使用内置代码中的变量。
- 变量可以进行计算与转换。
- 即使是同一脚本文件，绑定不同的对象从而产生了同名的变量，这两个变量以及变量代表的的数据也分别属于不同的对象，互相独立。因此使用变量前必须指明变量所属的对象。

②四种基本变量的定义、计算与转换

- 整形：int
- 浮点型：float
- 字符串：String
- 布尔：bool

③由四种基本变量组成的其他值类型变量

- 值变量：执行 变量 A= 变量 B 这段代码时，变量 A 会重新创建一个与 B 变量值相同的数据。对于 A 变量的修改无法影响到 B。
- 引用变量：执行 变量 A= 变量 B 这段代码时，变量 A 不会重新创建一个与 B 变量值相同的数据。对于 A 变量的修改可以影响到 B。
- 大部分变量都是值变量
- 二维向量展示

④变量的作用

- 变量除定义的过程外，必须在函数内进行数值的修改，内置代码中的许多变量在修改后，会直接对游戏内容产生实质性的影响。而自定义的变量则不会。因此，可以用自定义的变量来定义逻辑上的游戏数据，而后通过判断逻辑上的数据情况，来调整内置代码中的变量，对游戏的画面、音效等产生实际的影响。

04 GDScript 简单入门——函数

一、函数

①函数的概念

- 函数也叫方法或模块，是一套操作的集合。函数也包括自定义函数与内置函数。
- 大部分内置函数可以对游戏内容产生实质性的影响或在代码中起到传递数据的作用。少部分函数比较特殊。
- 一个函数除名称外，还包括了【输入信息、处理流程、处理结果】等关键信息。在程序中，一切数据的处理流程都要通过调用函数来实现。
- 函数调用一般必须指明调用对象

②内置虚函数的调用

- 虚函数是指没有实际处理流程的函数，这部分函数的内容可以由我们自行编写。而内置虚函数则是指 Godot 的制作组规定了特定名称却没有实际处理流程的函数。节点内的虚函数会在特定的条件下自动被触发。
- 条件 1：当节点本身或其周围节点状态发生改变时。【节点创建、节点入“树”、节点出“树”、节点死亡前、节点的子节点全部加入场景树·】
- 条件 2：当场景树的状态发生改变时，且节点自身处于“树”下。【画面刷新、物理引擎刷新、硬件设备输入】
- 条件 3：其他情况。
- 常用虚函数：_init,ready,_process,_physic_process

③函数中的变量

- 函数中也可以定义变量，各个函数都拥有一个独立的作用域，在一个函数内部定义的变量无法在另一个函数中直接访问变量名。
- 在函数运行结束后变量和变量所代表的数据会自动销毁。某些情况下，变量所代表的数据不会自动销毁，（后续教程提及）但变量一定会自动销毁。

05 GDScript 简单入门——IF 与 WHILE

一、IF

①if 的作用

- 使用 if 可以在函数中设置情况分支，起到控制数据处理流程的作用

• ②elif 以及 else

- elif 是 if 的补充，一个 if 可以有无数个 elif 对其进行补充。
- 当所有的 if 与 elif 判断失败时，程序执行 else 内部内容。一个 if 后只能有一个对应的 else。

③作用域

- 一个 if 与它对应的 elif 以及 else 都有各自独立的作用域，作用域内定义的变量互相独立。

二、WHILE

①while 的使用方法

- while 与 if 写法一致，但没有 elif 与 else
- 当 while 内部模块执行完毕时，程序会再次判断 while 是否成立，若成立则会继续执行 while 内容。

②while 的注意事项

- 避免死循环。要设置破坏循环条件的代码。
- while 也会产生一个作用域，效果同 if。

06 GDScript 简单入门——数组与 for

一、数组

①数组的概念

- 数组是一组数据的集合。在程序中负责批量处理数据。
- 数组中的元素可以包括各个类型的数据，也可以对数组内数据类型进行限定。
- 可以通过 数组名【数字】 的形式来访问数组元素，数字 0 代表数组的第一个元素。
- 数组可以通过调用函数来增加或去除内部数据。可以使用 size（）来获取数组内元素总数。
- 数组是引用变量
- 引用变量：执行 变量 A= 变量 B 这段代码时，变量 A 不会重新创建一个与 B 变量值相同的数据。对于 A 变量的修改可以影响到 B。

②for

- 使用 for 可以快速遍历数组

07 Input 单例与自定义单例

一、单例

①单例

- 单例是一个可以在任何一个脚本中对其进行直接访问的对象，分为内置单例与自定义单例。每个单例都是独一无二的对象。
- 内置单例不是节点，主要成员是各类 Server，开发者可以使用它们直接控制游戏程序的图形与音效等内容。此外，还包括了一些其他对象，它们涉及的范围包括网络、时间、电脑系统、输入等。
- 自定义单例必须是节点类型的对象，是开发者自定义的全局对象。

二、Input 单例

①Input 单例

- Input 是一个重要对象，它可以对玩家的按键情况进行反馈
- Input 的 Action 手动设置。
- Input 的常用函数介绍

②轮询

- 在 `_process` 或 `_physics_process` 中通过 Input 单例来获取按键情况，这种一秒数十次检测游戏输入情况的编码方式称为轮训。
- If+ 轮询 + 修改内置变量 = 游戏在玩家的控制下发生实质性的改变。

三、自定义单例

①自定义单例步骤

- 在项目设置中选择 Autoload。
- 选择脚本路径
- 点击添加

②自定义单例特征及用途

- 可以在任意一个脚本中对它们进行直接访问
- 用于记录全局变量

08 梦回 1972——Pong! （一）



一、Pong!

Pong 是世界上最古老的电子游戏之一。它的玩法非常简单，只需要两个玩家使用控制器控制不同的球拍，互相击球即可。

二、游戏场景安排

①游戏地图

- 地图以黑色为背景
- 在中央显示一道竖直白线，上下两侧显示白色墙壁
- 小球进入白色墙壁的范围后会发生移动角度的改变。
- 小球从左右两侧飞出后，会让小球回到原点

②玩家 1

- 接受玩家输入控制，能在一定范围内移动。

- 与小球接触后会击飞小球

③玩家 2

- 接受玩家输入控制，能在一定范围内移动。
- 与小球接触后会击飞小球

④小球

- 能够在地图中自由移动，碰到球拍与墙壁会发生速度方向的改变

⑤记分系统

- 能够记录左右两名玩家的分数
- 将玩家分数显示到屏幕上方

三、本期教程完成内容

①小球

②游戏地图

③计分系统

09 梦回 1972——Pong! （二）

一、本期教程的完成内容

①玩家 1

②玩家 2

11 场景内节点的协调工作—— get_node()

一、场景内的节点的组织工作

①使用场景内根节点的脚本组织场景内节点工作

- 使用脚本为节点拓展功能是一个非常常见的操作。但脚本过多会使得文件混乱。应尽可能减少一个场景内脚本文件的数目。因此一般的场景只在根节点添加脚本。
- 要在一个脚本内控制若干个节点的分工合作

②当小球接触玩家 2 的球拍时，球拍发出响声

- 本质：在 Area2D 节点的脚本内命令声音节点工作
- 前提：需要在 Area2D 脚本中获取声音节点对象
- 操作：声音节点对象调用播放音乐的函数，

二、get_node()与相关函数用法展示

①获取直接子节点

- `get_child()`

②获取直接父节点

- `get_parent()`
- `get_node("../")`

③获取多级子 / 多级父节点

- `/`

④获取所有的直接子节点

- `get_children()`

三、get_node()的优点与缺陷

①优点

- 可以直接获取到对象，然后调用对象内部方法或访问对象内部变量。操作与逻辑简便快捷。

②缺陷

- 使用时必须知道子节点的名称

- 使用 `get_node` 获取对象并直接调用对象内部方法或访问变量，如果对象、函数或方法不存在，程序会直接崩溃报错，对分块测试不友好。

12 GDScript 简单入门——信号与 await

一、信号

- 信号机制是 Godot 内一种调用函数的机制。
- 它需要将一个对象的“信号”与另一个对象的“函数”绑定。信号发射后程序会进行函数的调用。在绑定与调用过程中，如果双方对象，或目标方法不存在，则程序只会警告开发者，而不会崩溃。
- 如果使用信号去进行不同场景之间节点的调用，则可以保证每个场景都能够单独运行，方便对场景展开单独测试。

二、信号的使用演示

①信号的来源

- 内置信号与自定义信号
- 内置信号：在内置代码中定义，在一定条件下由 Godot 自动触发
- 自定义信号：需要由开发者在 GDS 中进行定义并触发

②信号的绑定

- 编辑器内绑定：方便快捷，容易操作。
- 代码绑定：适用的情况更丰富

③自定义信号的发射

- `emit_signal`

④await

三、信号的补充

①信号的连接与发射

- Godot 中所有的对象都可以进行信号的连接。
- 一个对象（A）内部的信号只属于这个对象（A），另一个对象（B）的同名信号在发射后无法触发 A 所绑定的方法。
- 每个对象内部都含有一个连接表，它记录了自身内部信号与方法的绑定情况，在发射时，对象会自动查询内部的表格。

13 GDScript 简单入门——对象与继承

一、对象与类

①对象与类

- 拥有自己属性、函数与信号的数据单位称为对象。比如节点，既具有内置属性又具有内置函数，也可以通过附加脚本的形式添加属性或函数。
- 我们可以称拥有相同属性、函数以及信号的对象们为一个类。

二、继承

①继承的概念

- 使一个A类拥有另一个B类的全部属性、方法与信号，则称A继承了B或B被A继承。（Node2D与Sprite）
- B类被A继承，称呼B为父类。A类继承了B，称呼A为子类。
- Node2D为父类，Sprite为子类。子类对象也是父类对象。Sprite也是Node2D类。

②Godot中的继承

- 内置继承：Godot中的节点都存在继承关系，Object是所有对象的基类
- 自定义继承：extends class_name 或 “路径”

③继承的优势

- "善"用继承，可以减少开发者的工作量。而不必编写重复代码。
- "善"用继承，可以方便代码的修改与维护。

④super用法

- super()
- super.函数名 ()

14 文档阅读

一、学习阶段性总结与文档阅读

①学习阶段性总结

- 在过去的教程中，我们已经掌握了 GDS 的大部分基础语法、开发了一个简单的 Godot 项目、并了解了一些基础的开发思想。
- 问题 1：对于 Godot 的细节操作仍不明白，对 Godot 游戏的运行机制仍然感到迷茫。
- 问题 2：对于 Godot 内数目庞大的内置变量、函数和信号等等内容却几乎一无所知。

②问题 1、2 的解决方案

- 问题 1：可以自主查阅 Godot 的官方在线文档或跟随他人的教程学习。
- 问题 2：学会阅读 Godot 内置的 API 文档，API 文档中记录了 Godot 内置的类、变量、函数和信号的相关介绍。

③阅读内置文档的意义

- Godot 的内置变量、函数与信号都被编写在 Godot 的内置文档中，它是 Godot 内置变量、函数与信号的说明书。
- 学会阅读与查询内置文档，可以大幅加快学习者自学的速度，可以自学掌握 Godot 中的许多类，是 Godot 入门中必不可少的一环。

二、阅读内置文档

①文档内容介绍

- 类名：类的名称
- 继承：当前文档的类继承的其他类，当前文档类拥有这些类的所有特征
- 派生：此类的子类
- 描述：文本介绍
- 属性：内置变量 变量类型 变量名
- 方法：内置函数 返回值类型 函数名（参数名：参数类型） 函数后缀
- 信号：内置信号 信号名 信号附加参数
- 枚举：表示特定数字的英文字母

②阅读文档的侧重点

- 在实战中阅读文档。
- 关注重点父类和常用节点。加速学习效率，快速搭建起知识骨架。

③检索文档的途径

①一般情况

- 根据节点或其他对象之间的继承关系，分析内容定位
- 根据数据类型，缩小检索范围
- 根据英文名称，缩小检索范围，猜测属性或函数的意义
- 耐心扒拉找

②特殊情况

- 根据函数后缀

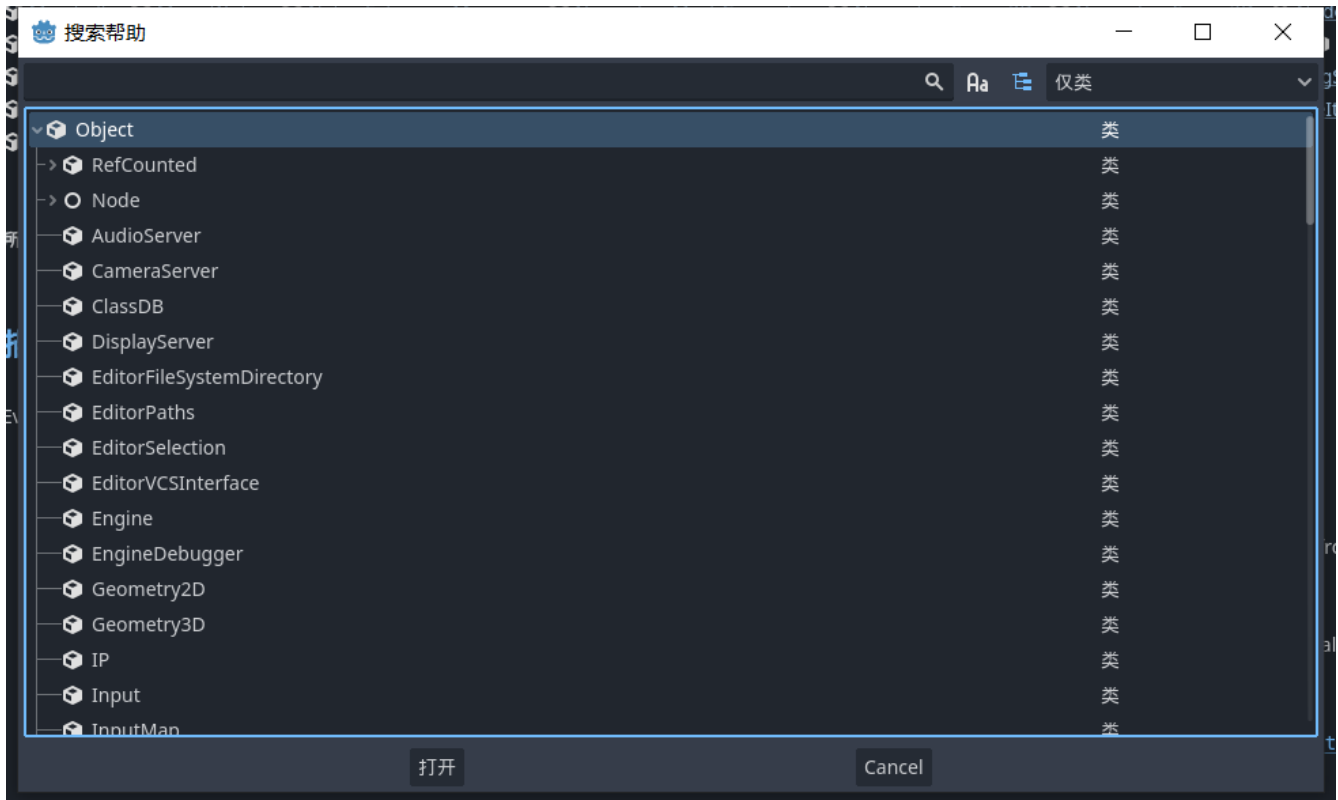
三、后续教程安排

- 后续教程会继续进行“知识点 + 实战项目”结合的形式来展开教学。
- 对于教程中提及的某些函数和属性将不再展开过多介绍，希望学习者可以自主查询内置文档了解。

▪

15 万类之父——Object

一、Object 的地位



Object 是 Godot 中所有类的父类，所有类都继承自 Object。Object 可以实现的功能，Godot 中所有的类也都可以实现。（某些内置单例无法调用 `set_script` 函数）

二、Object 的重要函数

- `_init()` `virtual` : 初始化虚函数，当一个对象生成时，此函数立即自动调用
- `_notification(what: int)` `virtual`: 接收通知虚函数，当此节点接收到 `notification` 的通知时此函数自动被调用。`what` 以数字的形式来告知此函数具体是接收到了那一个 `notification`。
- `connect/disconnect/is_connect`: 信号连接、取消连接、是否存在对应的信号连接。信号出现在 Object 内部，这代表 Godot 中所有的对象都可以进行信号连接，信号不是节点的专利。
- `emit_signal` : 发射信号。
- `call` 与 `call_defer` : 调用或在空闲帧中调用函数。对于空闲帧的解释会在后续教程中进行解释。
- `set_script(script)` : 设置此对象的 GDS 脚本。Godot 中大部分对象都可以设置脚本。

- `free ()` : 从程序中删除此对象，也就是将此对象从内存中移出。Godot 中所有对象都可以执行 free 功能。

三、记录 Object 的变量都是引用变量

16 节点之外的对象——RefCounted

一、RefCounted 概述

①RefCounted 的地位

- RefCounted 继承自 Object，它以及它的子类同样具有 Object 的所有功能。
- 它拥有众多的子类。RefCounted 内置了一个独特的引用计数器，要了解掌握这些子类，就必须先了解掌握 RefCounted 的引用计数器机制。

②RefCounted 对象的生成

- 类名.new() (类名由 class_name 进行自定义)
- load(“路径”).new() (也可以 preload(“路径”).new())

③RefCounted 的特征及作用

- 特征 1：内置引用计数器，当引用计数器归 0 时，此对象自动被程序删除。
- 特征 2：在 GDS 中不包含任何内置属性，仅有四个内置函数，也不必像节点一样要频繁的受场景树的控制，参与服务器的计算工作，相比于节点更加轻量。
- 作用：RefCounted 不但比 Node 占用更少的内存节约更多的电脑的算力、还可以进行内存的自动管理。非常适合用于制作游戏中数目庞大的数据对象。如仙侠游戏中的成百上千的随机 NPC 数据对象。

二、RefCounted 的重要子类

最重要：

- Resource——游戏文件与游戏程序的中转站，所有的图片、音频、视频文件都必须先转为 Resource，然后才可以被节点等对象使用。

了解即可，没有编程经验的学习者可以先不用了解：

- Astar2D/Astar3D 等可以用于构建游戏中的寻路数据。
- Thread/Mutex/SemaPhore 可以用于构建多线程。
- FileAccess/DirectoryAccess 可以用于加载读取计算机内的文件。
- 还可以使用某些子类进行 UDP、TCP 或 Websocket 的网络通信。
- StreamPeer 可以以字节单位对网络传输中收发的数据进行处理。
- 此外，它的子类还涉及了正则、JSON、游戏物理、自定义引擎编辑器等领域。

17 文件与程序的中转站——Resource 资源类

一、Resource 概述

①Resource 的作用

- Resource 是游戏文件与游戏程序的中转站。游戏项目内的大部分文件都要先转为 Resource 对象，然后才可以被游戏中的节点等对象使用。

②Resource 的继承关系

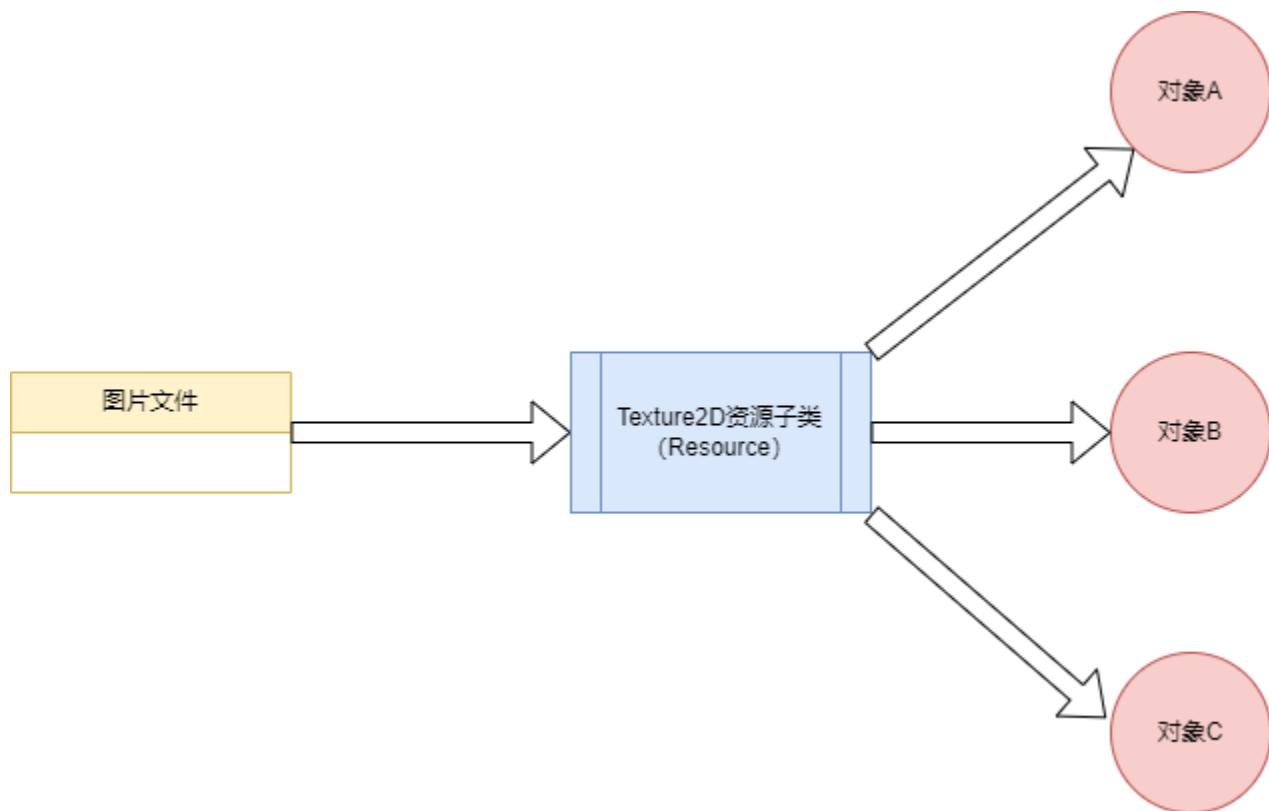
- Resource 对象继承自 RefCounted，RefCounted 继承自 Object，Resource 拥有二者全部的功能。
- Resource 可以进行信号绑定、脚本绑定等一系列 Object 可以进行的操作。
- Resource 也有内置引用计数器，计数器归 0 时，资源自动被删除。

③需要转为 Resource 的常见文件

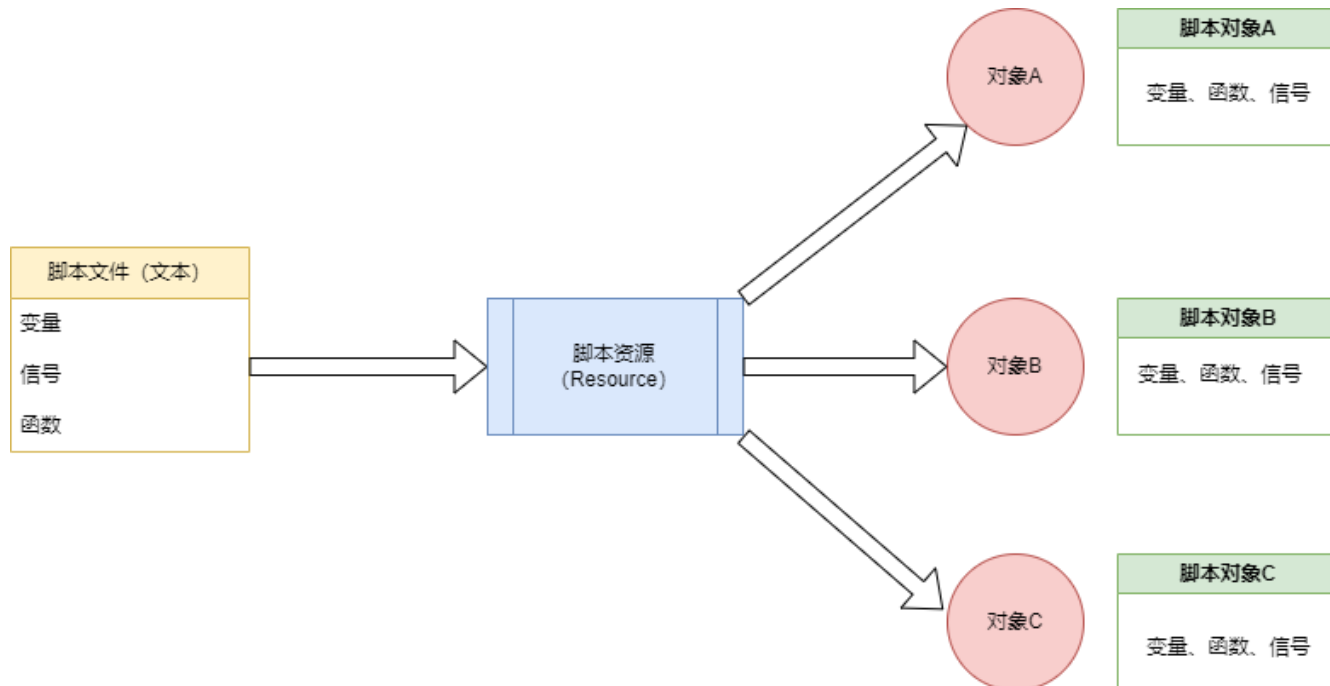
- 图片、3D 模型、音频、视频文件
- 场景文件
- 脚本文件

④资源供对象的使用的方式

- 多对象使用同一资源本身、适用于图片、3D 模型、音频、视频文件等。



- 资源经过再处理后，生成新对象，供不同对象使用。
- 脚本资源再处理生成的对象在 GDS 中不可见，这个新对象被编写在内置代码中，当我们使用 `set_script` 或绑定脚本的对象生成时它也将自动生成。



二、程序中 Resource 对象的创建

通过路径加载文件生成资源。

①资源随场景启动而自动创建

- 将资源文件（可以转化为资源的文件）拖入游戏项目中。
- 编辑器对资源文件进行导入，生成特殊文件以及资源文件对应的 import 文件。import 文件会指向特殊文件。
- 游戏启动后借由 import 文件加载特殊文件生成资源对象。

②load 与 preload

如果加载的文件已经被转化为资源，且此资源引用计数器不为 0，则在此加载此文件不会产生新的资源对象，而是返回一个原先资源对象的引用。

- `load("文件路径")` 或 `preload("文件路径")`

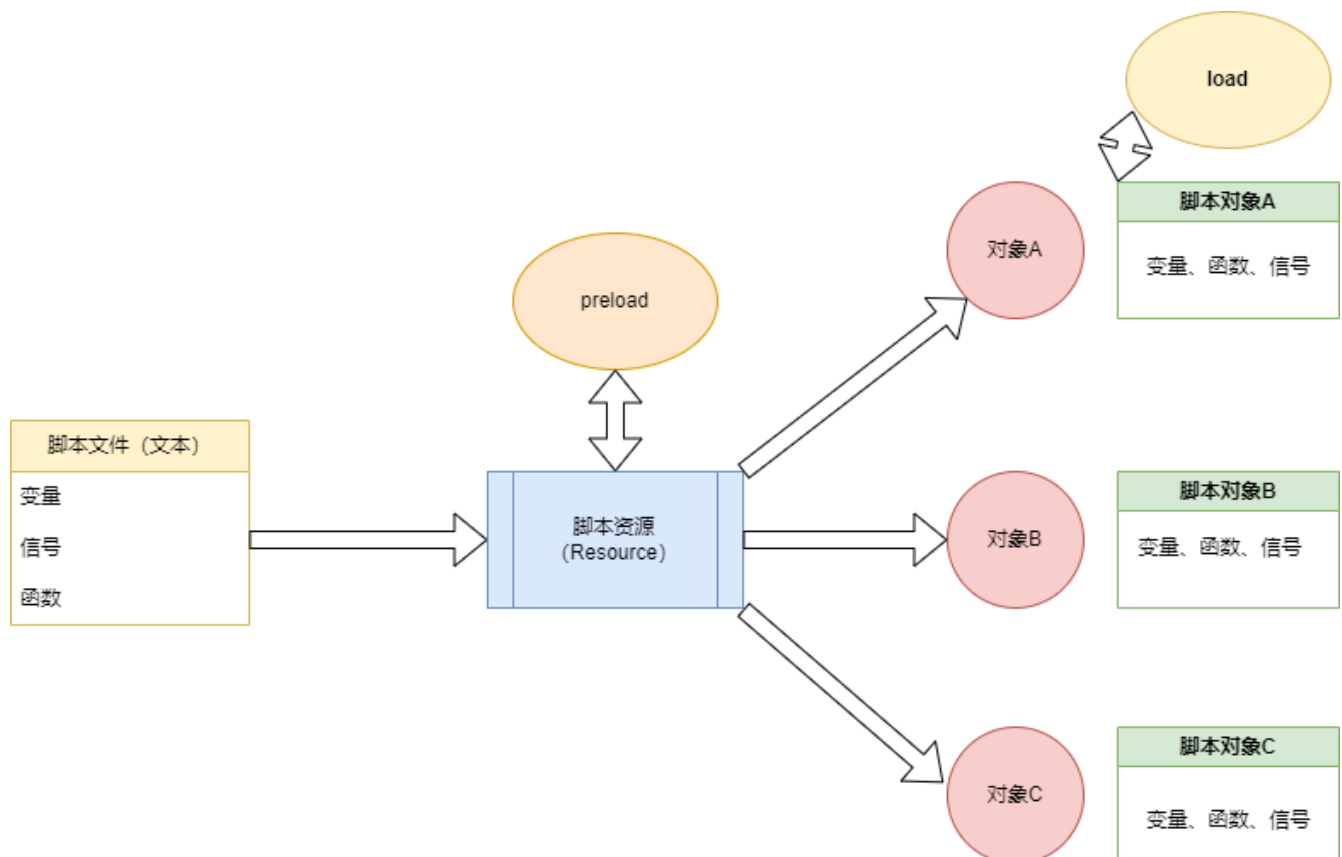
创建新对象。

③由类名.new () 创建，不会产生相同引用。

- 内置资源类：某些内置资源类也可以使用 new 以外的函数来创建资源对象。部分内置资源类内部拥有读取与保存文件的函数。
- 自定义资源类

三、load 与 preload 的区别

- load 与 preload 的参数都是文件路径，都是字符串形式，但 load 的参数可以是字符串变量。而 preload 则无法使用变量，这是由于二者执行的时机不一样所导致的。
- 当脚本文件转化为脚本资源时，转换部分的程序会自动翻找文件中是否出现了 preload 函数，若出现，则在脚本资源生成的同时进行 preload 资源的加载。这时候变量还没有出现，因此只能以文本字符串的形式来告知 preload 加载的内容。



- load 后可跟随字符串变量，字符串变量的值可以在程序运行时改变，如果改变变量，load 可以加载不同的文件。
- preload 后只能跟随固定的字符串，它不会在程序运行时改变。

四、资源保存的格式

资源对象是文件和程序的中转站，它不是简简单单的文件本身，一个资源对象可能是对一个或对个文件处理后产生的数据对象。这些处理可以是对若干图片的裁剪、压缩、拼接；一个资源对象，也可能是对节点某部分功能的记录与描述，比如 Pong 中区域检测时 shape 节点的形状就也是一个资源对象。要保存这些复杂的信息，Godot 必须准备单独的文件。

- tres ：文本资源格式。资源的内容以文本的形式储存，资源文件具有可读性。
- res ：二进制资源格式。简单来说就不是人话，但是程序加载速度更快。
- 某些资源会提供特殊格式来保存自己的数据。

五、Resource 对象的特征

- 加载路径产生资源时，如果原先的资源不消失，则不会产生这个文件的新资源对象，而只返回引用。在任意一处对此资源的修改，都将影响到所有使用此资源的对象。

18 程序启动第一步——加载项目设置文件

一、Godot 游戏程序启动的步骤

- ①程序初始化，游戏程序开始创建游戏进程，生成场景树与内置服务器。（使用 Godot 编辑器无法对这一步骤进行控制）
- ②游戏初始化，游戏程序加载项目设置，进行游戏程序基础信息的设置。
- ③将场景中的节点实例化，生成与之相关的脚本与资源，使其加入到场景树下，游戏正式开始运行。

二、项目设置可以设置的内容

①常规设置

- 对游戏项目相关的显示、音频、物理等方面进行设置。

②键位映射

③本地化（项目内文本翻译）

④Autoload

- 设置自定义单例

⑤Shader_Globals

- 添加全局 Shader 变量

⑥插件

- 添加或创建插件，拓展编辑器功能。

⑦默认导入设置

- 设置文件导入默认产生的特殊文件信息。

三、项目设置文件与单例

①project.godot

- 存放项目设置信息的文件，每个项目都拥有这个文件。加载项目设置就是加载这个文件。
- 除项目设置以外，class_name 自定义的类也会被写入这个文件。
- 此文件导出后会被放入 pck 文件中。

- 多人合作时，要小心此文件的覆盖问题。

②override.cfg

- 存放项目设置信息的文件。可以覆盖 project.godot 中重复的设置信息。但需要手动或使用代码创建。

③ProjectSettings

- 与项目设置相关的单例，可以使用代码在这个单例中设置项目设置的信息。并保存为 project.godot 或 override.cfg
- 使用代码修改的信息，不会对运行中的游戏造成实质性的影响。

19 节点的出现——场景文件的加载与保存

一、场景文件与场景资源

①场景文件

- 一组节点的集合。是节点加载和存储的基本单位。

②场景资源

- 场景文件对应的资源对象，再处理后可以生成对应的节点集合。并返回这个节点集合的根节点。

二、场景的实例化（加载）

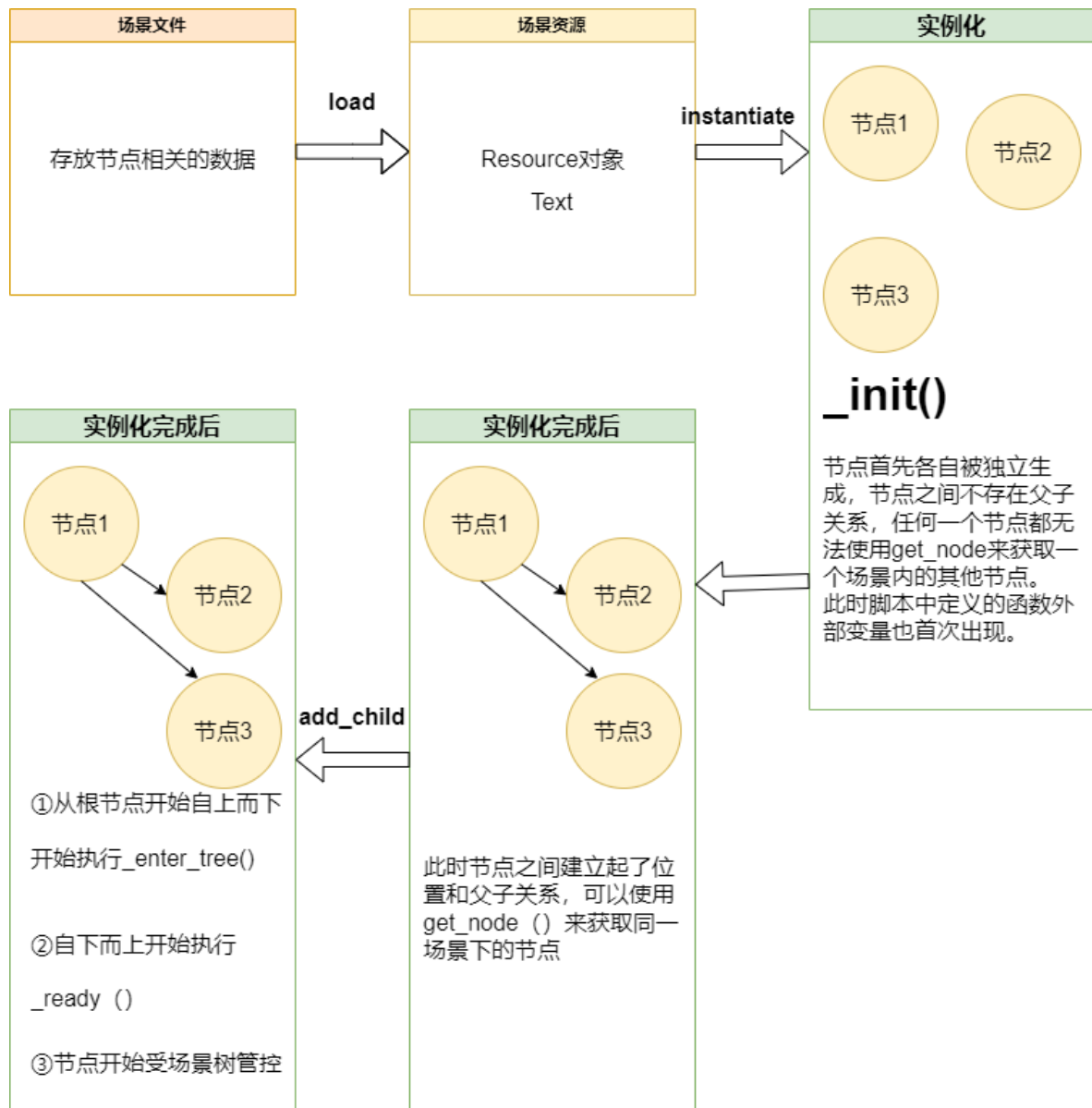
①场景加载的情景

- 游戏开始时，场景树自动将主场景实例化
- GDS 代码中将场景实例化

②实例化代码

- 场景资源的再处理就是场景的实例化。
- 一个场景资源调用 `instantiate` 将生成对应的节点集合，并且将这些节点的根节点作为返回值返回。
- 使一个处于场景树下的节点运行 `add_child()` 函数，并将新生成的根节点作为 `add_child` 的参数调用，则根节点就会成为这个节点的子节点，新生成的节点也将全部加入树下。开始行使功能。

③实例化的过程



③@onready 关键字

- 在函数外部使用@onready 来定义变量，可将变量的赋值拖延到 ready 执行的时刻

三、节点的 owner 属性与场景保存

①节点的 owner 属性

- owner 是一个节点类型的变量，用来表示某个节点。在一个场景文件实例化所产生的节点集合中所有节点的 owner 属性都指向这次实例化生成的根节点。

②场景的保存途径

- 编辑器中保存。
- 使用代码在游戏运行时将若干节点保存为场景。

③代码保存的注意事项

- 使用代码保存场景，只需要保存节点集合的根节点即可。

- 要对保存的节点集合中的节点设置 owner，将 owner 设为它们保存场景的根节点对象。

④保存代码示例

20 游戏管理者——MainLoop 与 SceneTree

一、SceneTree（场景树）与 MainLoop（主循环）

①场景树的作用

- 场景树是游戏的管理者，它负责进行 Godot 的内置服务器与节点的沟通工作。
- 内置服务器是 Godot 中的各种模块，包括计时系统、物理模拟系统、图像绘制系统等。
- 节点是 Godot 中最基本最常用的开发组件。

②主循环与场景树的关系

- 场景树类继承自主循环类，场景树是在主循环的基础上，对节点管理进行了扩写。
- 虽然存在主循环类，但游戏运行时只存在场景树对象。

二、MainLoop 的功能

①程序启动后，程序一定会创建一个主循环对象（虽然这个对象一般情况是他的子类场景树），它包含了**初始化、空闲帧同步回调、物理帧同步回调等方法**。这个类中不包含与节点相关的具体操作，我们使用 Godot 制作游戏时也几乎不会遇到需要自行编写 MainLoop 的情况。

②空闲处理与物理处理（Idle Process 与 Physic Process）

- 物理处理：Godot 游戏程序中会内置一个物理服务器，用于处理游戏世界内的各种物理计算。在每一次**物理计算之前**，这个服务器都会给予主循环一次参与计算的机会，这就是主循环中的物理处理。默认情况下，物理服务器一秒内会进行 60 次运算，因此主循环中的物理处理也会每秒进行六十次处理。
- 空闲处理：在 Godot 游戏程序相对“空闲”（当某些内置服务器运行结束的时候）的时候执行此处理。引擎会尽可能快的利用空闲时间来绘制新的游戏图像。

③节点中的空闲处理与物理处理

- `_process(delta)` : 空闲帧回调函数 `delta` 表示距上次空闲帧调用的时间
- `_physics_process(delta)` : 物理帧回调函数 `delta` 表示距上次物理帧调用的时间
- `call(函数名)` : 立即调用函数
- `call_defer(函数名)` : 到下个空闲帧调用函数

三、SceneTree 的功能

一、场景树

- 场景树继承自 Mainloop。
- 游戏运行时，将生成一个场景树对象，这个场景树对象将控制游戏中节点

二、场景树的功能

节点内部的 `get_tree()` 函数可以获取场景树

①组（Group）功能

- 组：将若干节点归纳为一组节点，使用场景树命令同组节点调用函数或修改属性。
- 节点的组操作：使得节点加入 / 退出组、判断此节点是否处于某小组内
- 场景树的组操作：命令同一组内的节点调用函数 | 判断某组是否存在 | 获取某组内的所有节点

②游戏暂停

- 当场景树进入暂停后，节点会根据自己的暂停模式来调节自己的状态。
- 节点的暂停模式：继承 | 正常暂停 | 暂停时运行 | 总是运行 | 从不运行
- 场景树暂停操作：修改 `pause` 属性

④场景树的其他功能

- 游戏退出
- 创建计时器
- 创建 Tween 动画
- 获取节点总数
- 设置网络连接
- 等等

三、场景树与 viewport 节点

当游戏开始加载主场景时，场景树会创建一个 viewport 节点，将主场景的根节点加载为 viewport 节点的子节点。下节课的内容将围绕 viewport 节点来展开介绍。

21 第一个节点——Viewport

一、第一个节点 Viewport

Viewport（视窗）节点是游戏运行时出现的第一个节点，当场景树第一次加载主场景时，游戏程序将先创建一个 viewport 节点并且将此节点添加到场景树下，然后再将主场景的根节点作为 Viewport 的子节点添加过去。场景树下的节点都是第一个 viewport 节点子节点。

二、Viewport 节点的功能

①Viewport 概述：

这个节点可以在屏幕中创建一个不同的窗口或在另一个窗口中创建子窗口。

游戏世界中出现的第一个 Viewport 节点将会创建一个 World2D 与一个 World3D 对象，它们分别代表一个独立运行的 2D 世界和 3D 世界，这个世界中将包括游戏运行时的物理状态、视觉场景与声音空间。Viewport 节点的子 Node2D 节点与子 Node3D 节点将分别参与两种世界的构成。

Viewport 节点的子 Camera2D/Camera 节点（2D 摄像机与 3D 摄像机）节点，可以调整游戏显示的区域，也会改变游戏世界中监听游戏声效的位置坐标。

②高阶应用：

- 访问 world_2d（world_3d）属性，直接对游戏世界造成影响或查询游戏世界中物理 / 视觉 / 声音元素的状态。

③常用应用：

- 双人游戏时的分屏效果

分屏效果就是用两个 Camera2D（Camera）节点，来显示同一个 World2D（World3D）对象内的不同区域内容。而一个 Camera2D 只可以影响到一个 Viewport 节点的显示范围，因此，我们可以创建两个代表相同 World2D 对象的 Viewport 节点。

补充：默认情况下，当我们手动新建一个 SubViewport 节点时，这个节点将会创建一个新的 World2D 对象，而不会去创建一个新的 World3D 对象。这将导致 Viewport 节点拥有一个独立的 2D 世界，而它代表的 3D 世界则是最近的上级 Viewport 节点代表的 World3D 对象。

- 输入事件的处理

④其他：

- 获取屏幕画面截图
- 设置 / 获取鼠标位置
- 等等（详情请查询文档）

三、输入事件处理（Viewport 节点最常用的功能）

①节点的_input与__unhandled_input函数

- _input与_unhandle_input都是输入事件处理函数，当我们点击鼠标、按下键盘和进行其他输入设备的操作时，一个对应的输入事件就会被创建，输入事件会在节点中进行传播。此时节点中的_input函数将根据传播的顺序依次被调用。
- 如果在_input传播中，输入事件未被处理掉，则在节点中再进行一次此事件的传播，此时_unhandle_input自动被调用。unhandle_input的含义即为未处理输入。
- 参数event可以代表鼠标点击、键盘按压、屏幕触碰、屏幕拖拽等多种输入事件，不同种类的事件被划分为InputEvent的不同子类，它们各自拥有不同的属性，在编写_input与_unhandle_input函数时，要先判断event的类型，再进行具体属性的处理。
- 鼠标左键点击代码示范

②Viewport对event的处理。

- 将事件设为已处理
- 创建input输入事件
- 创建unhandleinput事件
- 创建文本输入事件

22 游戏观测者——远程与调试器

游戏观测：

在实际开发中，我们可能会遇到各种各样的 bug，或者是遇上各种各样的优化问题。观测游戏运行时的状态将有效帮助我们解决这些问题。

.

一、远程

- 显示运行时场景树下的节点结构
- 显示节点相关的属性，可以进行修改与编辑。

二、调试

- 显示碰撞区域
- 显示寻路系统
- 等等

三、调试器

①错误

- 用红色文字提示代码书写错误，或反馈运行时发生的程序报错。
- 用黄色文字向开发者做出警告，被警告的代码一般是书写不够严谨的代码，他们不会直接破坏游戏的运行，具体的警告是否要处理，由代码编写者根据实际情况自行决定。
- 大部分情况下，错误的出现会破坏游戏的运行，使得游戏崩溃或出现图像和物理处理方面的 bug。

②性能分析器

- 以毫秒为单位，实时报告游戏中物理帧和空闲帧的间隔时间，以及物理、画面、音频、脚本处理所消耗的时间。

③visualprofiler

- 显示 CPU 与 GPU（处理器与显卡），进行画面渲染时每帧执行各项渲染功能时所消耗的时间。

④监视

- 显示游戏画面的刷新帧数，空闲帧的间隔时间和物理处理时间。
- 显示游戏的占用的内存，最大内存和消息队列最大占用内存。

- 显示游戏中的 Object、资源、节点和场景树之外的节点数目。
- 显示光栅、视频、物理、音频相关的状态情况。

⑤显存

- 显示游戏中各个资源占用的显存大小

⑥其他

- 帮助识别我们点击的 Control 类节点
- 可以显示此节点在场景树下的位置

⑦网络分析器

- 制作联网游戏时，显示 Godot 的流量数据。

四、断点与暂停

暂停：暂停运行中的项目，观测程序状态

断点：在脚本中设置断点，当代码运行到断点时进入暂停状态。进入断点后可以逐行运行代码，观察运行内部的状态。

第二部分结语：

- 第二部分的教程，主要是介绍了 Godot 运行的机制和 Godot 中节点以外的重要对象，这些知识，可以帮助我们更有条理的建立起 Godot 的框架。大部分人初学结束这部分的感受是云里雾里，这是正常的。
- 第三部分将会介绍具体的节点功能，这会比第二部分好理解很多，但是我建议我每介绍一个节点，大家自己去浏览一下这个节点的描述部分，并过目一下节点的属性和方法。

23 2D 游戏开发——CanvasItem 与 Node2D 节点概述

2D 游戏开发常用节点课程介绍：

这一部分的课程将会对 2D 游戏开发中的一些常用节点展开基础介绍。

每个节点都分三部分展开介绍。这三部分分别是：

- 节点描述
- 节点功能概括
- 初学者要掌握的基本功能。

这部分课程不会详细的介绍一个节点的全部功能，只会对节点基本功能进行一些常用的演示。如果想彻底地掌握一个节点的功能，还是建议在入门 Godot 以后去自行查阅相关的文档，文档的内容是最详细的。我们的课程主要介绍的是入门所必须掌握的功能。

一、CanvasItem（画布项）节点

①CanvasItem 概括

CanvasItem 节点继承自 Node，可以用这个节点进行 2D 图画的绘制，它有 Node2D 和 Control 两个子类。Node2D 用于制作 2D 游戏对象，Control 用于制作游戏界面。

②CanvasItem 功能

- CanvasItem 节点提供了众多用于绘制图像的函数。
- 提供了部分与图像渲染相关的属性
- 允许使用者为此类节点（子节点）添加 shader

③初学者要掌握的部分

- visible 属性
- Modulate 属性
- 渲染顺序与 Order 属性

二、Node2D 节点

①Node2D 节点概括

Node2D 节点继承自 CanvasItem，它拥有非常丰富的子类，这些子类涉及到了 2D 游戏的图像显示、物理模拟、寻路系统、2D 音效、2D 灯光等多个领域的内容。

②Node2D 功能以及作用

- 提供了 2D 相关的位置 / 旋转 / 缩放 / 倾斜等属性。
- 提供 2D 矩阵属性，可以通过修改矩阵实现一些高级的画面操作。
- 内置了部分函数，可以用于修改上述属性。

③初学者要掌握的功能

- 了解 Node2D 以及 CanvasItem 类节点的传递性
- 了解 Node2D 的位置 / 旋转 / 缩放 / 倾斜等属性

24 2D 游戏开发——Sprite2D 与 AnimatedSprite2D

一、Sprite2D 节点

①节点描述

显示图片的节点

②节点功能

- 显示图片纹理。
- 提供了可以对纹理进行位置偏移、翻转等操作的属性。

③初学者掌握功能

- 学会如何使用编辑器 / 代码为该节点添加图片纹理。
- 了解部分简单属性。

二、AnimatedSprite2D 节点

①节点描述

通过切换图像，显示不同图片的形式来显示游戏动画的节点

②节点功能

- 具有播放动画与停止动画的方法。
- 具有动画播放相关的部分属性，修改这些属性可以影响到动画播放的细节。

③初学者掌握功能

- 了解播放与停止动画的方法和常用的属性
- 学会如何使用编辑器设置动画

25 2D 游戏开发——Body2D 类节点与 Area2D 节点

一、Body2D 类节点与 Area2D 节点

①节点描述

现实世界中，我们肉眼所见的各种物体，如篮球、鸡和墙壁除了在视觉上可见之外，在触觉上也都是互相可感的，我们既可以用双眼看见鸡与篮球，也可以用双手感知鸡与篮球，并且我们的双手也不会凭空穿入鸡与篮球的肉体。现实世界中的逻辑是可见即可感。

而在游戏世界中，视觉与触觉，即渲染与物理两个要素是两个互相独立的部分。Sprite2D 可以提供视觉上的游戏对象，却无法在游戏中产生“触觉”即物理意义上的对象。想要让游戏世界中出现可以感知的，且占据一定空间体积的物理对象我们就要借助其他节点，Body 类节点主要就是用于制作游戏世界中的物理对象。而 Area 节点除了可以用于检测感觉这些物理对象以外，还可以对一个区域内的物理属性、音轨等内容造成影响。

上述二者均继承自 CollisionObject2D 节点，CollisionObject2D 节点是 2D 世界中物理对象的基类（父类），它可以容纳任意数量的 2D 碰撞形状，这些 2D 形状用于定义 Body 代表的实体形状和 Area 节点所规定的范围。

②节点功能

Body2D 类：

- StaticBody2D：用于制作相对静态物理对象，一般来说静止的墙壁或者是移动方式较为固定的平台均可使用此对象制作。
- RigidBody2D：拥有质量、重力缩放、受力情况等多种物理属性，可以用于制作不受代码控制而能够自由移动物理对象。
- CharacterBody2D：内置了许多与移动相关或位置相关的函数，· 用于制作受代码控制的物理对象。

Area2D 节点：

Area2D：可以检测并获取他区域内的 Body2D 和 Area2D 节点，可以为区域内施加额外的物理属性，或修改区域内的音轨情况。

③初学者需要掌握的功能

- 学会区分三种不同 Body 的区别，简单了解物理材质的作用，理解碰撞层与遮罩层的意义并掌握其设置方法。
- 了解掌握 CharacterBody2D 节点的移动代码。
- 掌握 Area2D 的区域检测函数和信号的使用。

26 2D 游戏开发——RayCast2D 与 ShapeCast2D

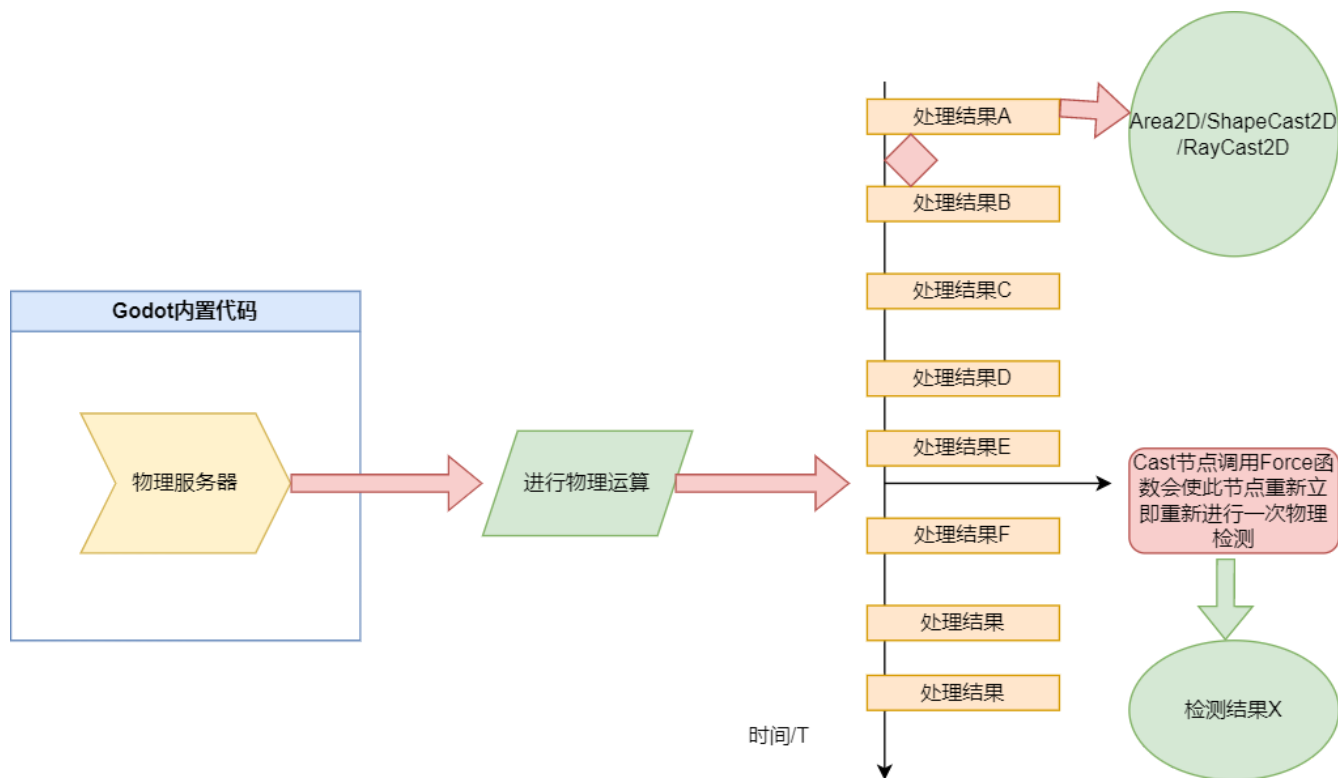
一、RayCast2D（射线投射检测）与 ShapeCast2D（形状投射检测）节点

①节点功能

RayCast2D 与 ShapeCast2D 节点可以检测获取 2D 世界内的 Body2D 与 Area2D 节点。这两个节点都拥有一个靶坐标，RayCast2D 会从节点坐标出发，创建一条到靶坐标的射线，检测这条射线上距离节点坐标最近的 Body 或 Area。ShapeCast 则会向靶坐标处创建若干连续的 Shape 以检测距离节点坐标最近的 Body 或 Area。

②初学者需要掌握的功能

- 认识 RayCast2D 和 ShapeCast2D 的常用属性
- 区分这两个节点和 Area2D 节点的区别



27 2D 游戏开发——TileMap 节点

一、TileMap 节点

①节点描述

进行 2D 贴图平铺的节点，多用于制作游戏的地图。

②节点功能

节点本身：

- 开发者可手动使用此节点内 tileset 对象所提供的图块进行地图的绘制工作。
- 开发者可手动设置绘制图块的图层分布。
- 开发者可使用代码获取、分析、设置节点绘制出的图像情况。

tilesset：

- 设置图片中图块的划分方式，创建或删除图片中的图块。
- 为图块创建物理碰撞相关的形状、layer、以及 mask 属性。
- 为图块创建游戏导航功能所需的相关功能。
- 为图块创建自定义数据
- 可以将多个图块整合，设置为 Pattern 或地形（Terrain）图块。
- 可以将场景设置为“图块”。

③初学者需要掌握的功能

- 掌握最基本的图片的划分和创建图块的方式。
- 掌握为图块创建物理碰撞形状的方式。
- 掌握 Pattern 和场景图块的设置方式。
- 大致了解 Terrain 地形图块的设置流程即可。

28 游戏界面开发——Control 节点介绍

一、Control 节点

①节点描述

Control 节点继承自 CanvasItem，拥有 CanvasItem 节点所具有的所有特征，与 Node2D 节点存在许多相同类似的属性。Control 是所有界面类节点的父节点，Control 拥有非常繁多的属性、方法以及信号，这些内容都深刻影响着 Control 子类节点的工作。此外 Control 节点还定义了一套比坐标系更加复杂的布局系统，这套布局系统可以使得节点树中父 Control 类型的节点更加便利的管控子 Control 节点的位置、大小等显示相关的属性。

②节点功能

- 提供了一套专属于界面类节点的布局方案，以及配套这套方案的属性与函数。
- 提供了特殊的输入处理函数 `_gui_input` 和一些关于输入处理的函数。Control 类节点的特有输入处理较为特殊，与渲染顺序、图层、可见性、聚焦等属性高度相关。
- 提供了界面类节点实现拖拽功能的虚函数。
- 提供了与语言翻译转换（本地化）、快捷切换界面节点等功能相关的属性与函数。
- 提供了 Theme（主题）属性和修改此属性的一系列函数，可以在其中规定各种 Control 类节点的显示效果等参数。节点树中的子 Control 节点默认继承父 Control 节点的主题属性。
- 提供了移动鼠标的函数。
- 提供了设置自动提示文本的属性。

③初学者需要掌握的功能

- 了解锚点和锚偏移的基本概念，掌握 Control 类节点布局属性的使用。
- 掌握 `_gui_input` 的使用，以及 Control 类节点处理特殊输入时与渲染顺序、可见性、图层、聚焦等属性的关系。

29 游戏界面开发——Container 节点

一、Container 节点

①节点描述

Container 节点是 Control 类节点的一个重要子类，当新的 Control 类节点加入成为 Container 节点的子节点时，Container 类节点会发出 `pre_sort_children` 信号，之后，Container 节点会根据代码设置的布局方式，排布子 Control 节点。

②节点功能

- 通过代码以自定义的布局方式，排布子 Control 节点。
- 以现有节点所规定的布局方式，排布子 Control 节点。

③初学者需要掌握的知识

- 认识并学会简单使用常见 Container 节点即可。
- 了解如何使用 `pre_sort_children` 以自定义布局方式。

30 游戏界面开发——常用 Control 节点介绍

涉及节点：

- Button
- TextureButton
- Label
- TextureRect
- TextureProgressBar
- VideoStreamPlayer
- LineEdit
- TextEdit
- SpinBox
- ColorRect

31 第三部分补充——三种常用节点

- **Timer:**

计时器节点，可用于完成游戏中的计时功能。如技能 cd、关卡通关计时等。

- **AnimationPlayer:**

动画播放器节点，更加高级的动画节点。可以在时间轴上设置属性的改变过程，以及在某个时间点设置函数的调用。

- **CanvasLayer:**

图层节点，可以在游戏世界中创建真正意义上不同的图层。

对于 CanvasItem 类节点而言，高图层的对象在绘制时注定覆盖低图层的对象，因此高图层的 Control 类节点如 mouse_filter 属性为 stop，则可以阻断传往低图层 Control 节点的输入事件。

对于 Camera2D 节点而言，Camera2D 节点仅能影响到自己所在图层上的显示区域的位置，而无法影响到其他图层的显示区域的位置。

32 入门项目实战——入门项目准备

一、入门项目介绍

第四部分的教程将教会大家如何制作一个最简单的 2D 平台跳跃游戏，但由于 Up 主的原因，最后一部分的质量并不是很好。教程内容也完成有限，想要呈现出更好效果的话，需要大家在完成教程后，进一步对项目内容进行打磨

二、项目素材获取途径

最后一部分的素材主要选于 Itch.io。

网址分别是：

远程怪物：<https://chierit.itch.io/elementals-leaf-ranger>

近战怪物：<https://chierit.itch.io/elementals-fire-knight>

主角：<https://rvros.itch.io/animated-pixel-hero>

地图：<https://anokolisa.itch.io/moon-graveyard>

UI：<https://adwitr.itch.io/pixel-health-bar-asset-pack-2>

此外也可以加群获取，群号在视频简介的下方

如果你觉得 UP 主的教程做的还可以，可以给予 UP 一些经济上的支持，也可以点赞投币加三连，或者怂恿别人直接扫码 0.0 我是一个非常贪财的 UP。

推荐使用微信支付



亚托克斯(**翰)



微信支付

33 入门项目实战——敌人场景

敌人类别：

远程弓箭手

近战刀手

通过敌人特征来梳理节点：

- 拥有基本的 2D 行走能力 ——> Node2D 类节点
- 拥有一具游戏世界内的物理身体 ——>CharacterBody2D(RigidBody2D)
- 拥有攻击功能，可以对主角造成伤害 ——>Area2D 节点
- 拥有匹配各个动作的动画功能 ——>AnimatedSprite2D 节点
- 拥有播放音效的功能 ——>AudioStreamPlayer2D 节点
- 拥有不跌落悬崖的检测功能 ——>RayCast2D 节点（Area2D）
- 拥有检测周围是否存在主角的功能 ——>RayCast2D 节点（Area2D）

34 入门项目实战——远程敌人与近战敌人

输入正文或"/"插入内容

34 入门项目实战——状态机与近战敌人场景

状态机：

①普通编码模式的缺陷：

随着游戏中角色功能的增多，开发者需要在编码中添加更多的 if，以保证角色在合适的情况下可以触发特定的功能。在这种情况下，在一个脚本内设计一套逻辑严密、条理清晰的代码是一件非常困难的事情。

为了解决这种问题，将一个角色的功能划分为不同的状态，再对各个状态进行编码，可以有效减少编码时的压力。这虽然会要求开发者多编写一些代码的行数，但却更容易让开发者设计出一套逻辑严密、条理清晰的代码，方便开发者后续实施对代码的修改和扩展工作。

②状态机的基本思想：

状态机是一种编码方式，也是一种编程思想，它本身不具备固定的格式、函数、以及属性，具体的编码内容取决于项目的规模和具体需求，但状态机的基本思想却是固定的。

状态机的代码主要由两部分组成，其一是描述游戏对象功能的各种状态代码，其二是对这些状态进行管理的管理代码。状态代码主要包括三部分内容，这三部分分别是：

- 进入状态时触发的代码，将游戏对象切换为新的状态。
- 状态中运行的代码，执行状态功能，检测是否要进行状态的转换。
- 退出状态时触发的代码，消除此状态对游戏对象的影响。

管理代码的功能主要是存放各类状态代码，以及给予这些代码一个状态转换的函数。在状态转换的函数中，老状态会运行退出函数，而后将游戏角色的状态切换为新状态，再命令新状态调用进入函数。

35 入门项目实战——游戏地图的搭建

输入正文或"/"插入内容

36 入门项目实战——开始界面的搭建

输入正文或"/"插入内容